Actividad API REST (Ionic + Angular Standalone) — entrega final

Jorge González Gómez · 2º DAM · Desarrollo de Interfaces · 26-10-2026 (versión 2)
GitHub: github.com/JorgeGonzalezDI/Desarrollo-Interfaces-DAM-2/tree/main/03-API-REST-Productos · Vercel: https://desarrollo-interfaces-dam-2-qeef.vercel.app

> Entrega completa: las 10 peticiones están hechas. Esta versión la he **rehecho a mano, paso a paso**, para entender cada pieza (servicio, signals, rutas con lazy loading) y no solo copiarla. Todo el código está comentado en GitHub, que es donde se ve cada cambio.

Peticiones hechas

- **Base:** modelo, servicio con `HttpClient`, rutas, Inicio y tabla de dummyjson con estados de carga/error/reintento.
- **1.** Ancho × alto en la tabla.
- **2.** Stock valorado (unidades × precio − descuento).
- **3.** Botón «Volver a inicio».
- **4.** Página About con reseña y enlace a GitHub.
- **5, 7 y 8** explicados abajo (troubleshooting, `app.config.ts`, ramas y Vercel).
- **6.** Paginación con `limit` y `skip`.
- **9.** Botón de modo oscuro.
- **10.** Vista de cards, con selector Cards / Tabla.

Lo más interesante: con HttpClient no hay async/await

`HttpClient` devuelve un **Observable**, no una Promise: la petición no se envía hasta que alguien se suscribe. Por eso el servicio solo «prepara» la petición y la página se suscribe con `next` / `error`.

```
// product.service.ts
private http = inject(HttpClient);                        // inyección de dependencias
getProducts(limit = 10, skip = 0): Observable<ProductsResponse> {
  return this.http.get<ProductsResponse>(this.apiUrl, { params: { limit, skip } });
}

// productos.page.ts
this.productService.getProducts(this.pageSize, skip).subscribe({   // aquí se envía la petición
  next: (response) => { this.products.set(response.products); this.loading.set(false); },
  error: () => { this.error.set('No se han podido cargar los productos.'); this.loading.set(false); },
});
```

Petición 2: stock valorado

```
stockValorado(p: Product): number {
  const bruto = p.stock * p.price;                        // unidades * precio
  return bruto - bruto * (p.discountPercentage / 100);    // menos el % de descuento
}
```

Ejemplo: 99 uds × 9,99 € = 989,01 €; con un 10 % de descuento → 890,11 €.

Petición 6: paginación

La API devuelve 30 productos por defecto, pero hay 194. Pido de 10 en 10 con `limit` y `skip` (`skip = (página − 1) × 10`). El número de páginas es un `computed`, que se recalcula solo cuando llega `total`.

```
pagina = signal(1);
totalPaginas = computed(() => Math.ceil(this.total() / this.pageSize));   // 194 / 10 → 20

siguiente(): void {
  if (this.pagina() < this.totalPaginas()) {
    this.pagina.update((p) => p + 1);     // nuevo valor a partir del anterior
    this.loadProducts();                  // vuelve a pedir con otro skip
  }
}
```

Petición 9: modo oscuro

Ionic trae paletas de color. Con `dark.class.css` el modo oscuro se activa al poner la clase `ion-palette-dark` en `<html>`. Lo hace un servicio compartido por las páginas, que además recuerda la elección en `localStorage`.

```
@Service()
export class ThemeService {
  dark = signal(false);                                  // true = modo oscuro
  toggle(): void { this.aplicar(!this.dark()); }
  private aplicar(oscuro: boolean): void {
    this.dark.set(oscuro);
    document.documentElement.classList.toggle('ion-palette-dark', oscuro);   // Ionic cambia los colores
    localStorage.setItem('modo-oscuro', String(oscuro));                     // se recuerda al recargar
  }
}
```

Petición 10: vista de cards

Un signal `vista` decide si se enseñan cards o la tabla (selector con `ion-segment`). Las cards van en una rejilla que se adapta sola al ancho: `grid-template-columns: repeat(auto-fill, minmax(220px, 1fr))`.

Petición 7: ¿por qué existe `app.config.ts`?

En Angular standalone ya no hay `AppModule`. La configuración global (router, Ionic y `provideHttpClient()`, necesario para poder inyectar `HttpClient`) va en un objeto `ApplicationConfig` en `app.config.ts`, y `main.ts` arranca con `bootstrapApplication(AppComponent, appConfig)`.

Petición 8: de la rama desarrollo a main

Trabajo en `desarrollo`; cuando algo está probado lo paso a `main`, que es la rama que publica Vercel:

```
git checkout desarrollo → git add . → git commit -m "..." → git push origin desarrollo
git checkout main → git merge desarrollo → git push origin main → git checkout desarrollo
```

Petición 5: troubleshooting

- **No compila `'@ionic/angular/standalone'`.** Mi proyecto usa Ionic 9 y esa ruta ya no existe. Solución: importar de `'@ionic/angular'`.
- **Se queda en «Cargando productos…» para siempre.** Angular 22 no usa zone.js y no detecta cambios en variables normales dentro del `subscribe`. Solución: guardar el estado en signals (`signal()` + `.set()`).
- **404 al recargar `/productos` en Vercel.** Es una SPA y solo existe `index.html`. Solución: `vercel.json` con un rewrite de todas las rutas a `index.html`.